UNIT 9 75 min

Dictionaries

Store values under named keys: create, read, update, safely look up, loop over, and nest dictionaries.

By the end of this unit you will be able to

- Create a dictionary and read, add, and update values by key
- Use `.get()` to look up a key that might be missing
- Loop over keys, values, and key-value pairs with `.items()`
- Build a dictionary by counting items in a loop
- Work with a dictionary nested inside another dictionary

Key-value pairs

A **dictionary** maps **keys** to **values**. Instead of remembering that item 2 is Ben's score, you look up `scores["Ben"]`. Keys must be immutable (strings and numbers are the usual choices). Values can be anything.

dict_basics.py

```
student = {
    "name": "Maya",
    "age": 16,
    "grades": [91, 88, 95]
}

print(student["name"])
print(student["grades"][0])
```

Adding and changing values

Assigning to a key that exists updates it. Assigning to a new key adds it.

dict_update.py

```
scores = {"Ana": 90}
scores["Ben"] = 85      # new key: added
scores["Ana"] = 95      # existing key: updated
print(scores)
```

Safe lookups with get()

Reading a missing key raises `KeyError`. `.get(key, default)` returns the default instead, which is the safe choice when the key might not exist.

get_safe.py

```
prices = {"apple": 0.50, "pear": 0.75}
print(prices.get("apple", 0))
print(prices.get("kiwi", 0))
```

> **Watch out**
> A missing key with `prices["kiwi"]` raises `KeyError: 'kiwi'`. Use `.get()` when the key is not guaranteed to exist.

Looping over a dictionary

`.keys()` gives the keys, `.values()` gives the values, and `.items()` gives both as pairs. Looping over the dictionary directly visits the keys.

Trace: loop over items TRACE

```
grades = {"Ana": 90}
grades["Ben"] = 85
grades["Ana"] = 95
for name, score in grades.items():
    print(name, score)
```

Step	Line run	Variables after the line	Output so far
1	1	`grades = {'Ana': 90}`	—
2	2	`grades = {'Ana': 90, 'Ben': 85}`	—
3	3	`grades = {'Ana': 95, 'Ben': 85}`	—
4	4	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ana', score = 95`	—
5	5	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ana', score = 95`	`Ana 95`
6	4	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ben', score = 85`	`Ana 95`
7	5	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ben', score = 85`	`Ana 95 | Ben 85`

Counting with a dictionary

A very common pattern is counting how often each item appears. Start empty, then for each item, use `.get(item, 0) + 1` to add one to its count.

counter.py

```
words = "the cat saw the dog and the bird".split()
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
print(counts)
print("'the' appears", counts["the"], "times")
```

Nested dictionaries

A dictionary can hold other dictionaries or lists. Chain the keys to reach deeper values. Real data (JSON from web APIs) looks exactly like this.

nested.py

```
class_roster = {
    "period_1": {"teacher": "Ms. Lee", "students": ["Ana", "Ben"]},
    "period_2": {"teacher": "Mr. Cruz", "students": ["Cy", "Dee", "Eli"]}
}

print(class_roster["period_2"]["teacher"])
print(len(class_roster["period_2"]["students"]))
```

Predict the output THINK FIRST

Keys decide what is found. Watch for duplicates and for missing keys.

```
d = {"a": 1}
print(d["a"])
```

Answer: `1`

```
d = {"a": 1}
print(d["b"])
```

Answer: `KeyError`

```
print({"x": 1}.get("y", 0))
```

Answer: `0`

```
d = {"k": 2}
d["k"] = 5
print(d)
```

Answer: `{'k': 5}`

```
print(len({"a": 1, "a": 2}))
```

Answer: `1`

The second "a" replaces the first. Keys are unique.

```
print(1 in {"a": 1})
```

Answer: `False`
`in` checks keys, not values.

```
d = {"p": {"age": 3}}
print(d["p"]["age"])
```

Answer: `3`

Practice

▼ **Exercise 1 — Inventory**

Start with `stock = {"pencil": 4, "pen": 2}`. Add 3 pens, add 5 erasers (a new key), then print the whole dictionary.

▼ **Show solution**

Solution

```
stock = {"pencil": 4, "pen": 2}
stock["pen"] = stock["pen"] + 3
stock["eraser"] = 5
print(stock)
```

▼ **Exercise 2 — Most common**

Using the counts from the counter example, find the word with the highest count and print it. Test with the sentence `"a b a c a b"`.

▼ **Hint**

Loop over `counts.items()` and keep track of the best word and count seen so far.

▼ **Show solution**

Solution

```
words = "a b a c a b".split()
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1

best_word = None
best_count = 0
for word, n in counts.items():
    if n > best_count:
        best_word = word
        best_count = n
print(best_word, best_count)
```

Discussion

1. When is a list of pairs a better choice than a dictionary, and when is it worse?
2. Why do you think dictionary keys must be immutable? What would go wrong if a key could change after it was stored?
3. Think of a real system you use (a school portal, a game, a store app). What are its keys and values?

▼ **Facilitator notes**

Ask students to compare `d[key]` and `d.get(key)` on a missing key. Note that `.get(key)` with no default returns `None` instead of raising an error.

Key terms

Dictionary

A collection that maps unique keys to values, written in curly braces with `key: value` pairs.

Key

The label used to look up a value in a dictionary. Keys must be immutable and unique.

Value

The data stored under a key. Values can be any type.

.get()

A dictionary method that returns the value for a key, or a default if the key is missing.

.items()

A dictionary method that yields key-value pairs, usually unpacked in a loop as `for k, v in d.items()`.

KeyError

The error raised when reading a key that is not in the dictionary.

Nested dictionary

A dictionary that contains another dictionary or list as a value.

Counter pattern

Using a dictionary to count occurrences: `counts[item] = counts.get(item, 0) + 1`.

See all terms in the glossary →